Advanced Programming

Árpád Tamási · ELTE Faculty of Social Sciences

SESSION 7 — 20 OCTOBER 2026

SESSION 7

Today

60 minutes active, 30 minutes consultation.

Halftime · Ideas · Two weeks on your project · gstack · Impeccable · Homework

Halftime

Halftime

Half the semester is behind us. Your projects are different sizes, and they are in different places.

Some of you are done. Then make it bigger.

Some of you aren't. Then use the time to get there.

SESSION 7 HALFTIME

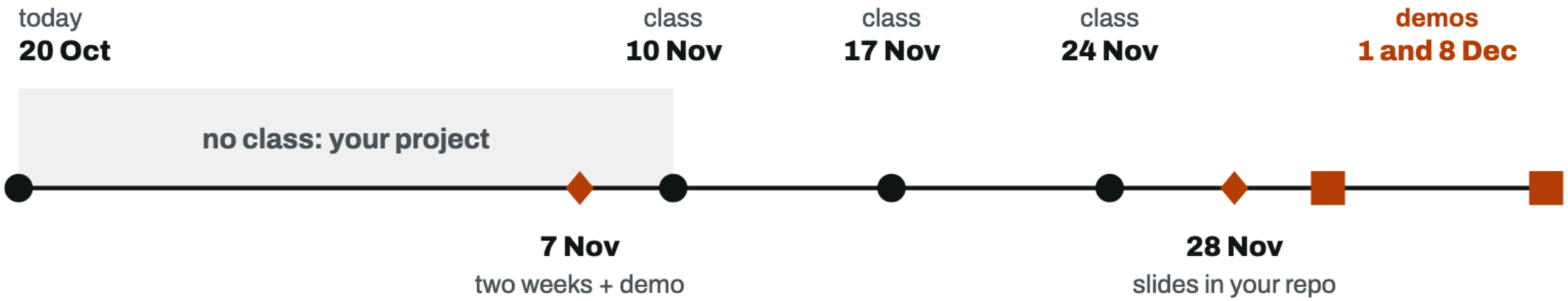
If you're done: make it bigger

1. **If you built something you really use,** you already know what's missing. Build that.
2. **If you don't use it yet,** ask yourself: what would it take for me to use it? Build that.
3. **If you would never use it,** find someone who would, and ask them what is missing. Or ask gstack: `/office-hours`.

SESSION 7 HALFTIME

Your project is the homework now

For the rest of the semester, the homework is your project moving forward. Use the time: six weeks to the first demos.



Ideas

Ideas

A database · users and login · a real interface · connecting it to AI · and more

Any of these, or something else. A few words on each.

A database

WHAT IT'S FOR

- Data that outlives one run of your program.
- Several people, or several devices, using the same data.
- Finding things fast in a lot of data.
- Changes that must not half-happen: a payment, a booking.

WHAT IT ADDS

- A schema to design, and migrations when it changes.
- Somewhere for it to run: a server or a service, with a cost or a free-tier limit.
- Backups.
- Who may read and write it, and its keys kept out of git.

Four kinds of database

	RELATIONAL (SQL)	NOSQL: DOCUMENT	NOSQL: KEY-VALUE	VECTOR STORE
Data	tables with fixed columns, rows linked by keys	documents like JSON, fields can differ	one value for each key	vectors: numbers that stand for the meaning of a text or an image
Query	SQL, joining tables	by the fields of a document	only by the key	by similarity: “the closest ones to this”
Examples	SQLite (one file), PostgreSQL	MongoDB, Firestore	Redis, the browser’s `localStorage`	pgvector (in PostgreSQL), Chroma
Fits	clear structure, things that relate	changing shape, data synced to an app	settings, sessions, caches	search by meaning; giving a language model the right documents

Users and login

WHAT IT'S FOR

- Each person sees their own data.
- Knowing who did what: who added it, who changed it.
- Different people may do different things: someone edits, someone only looks.

WHAT IT ADDS

- Accounts: signing up, a forgotten password, deleting an account.
- Personal data: from now on you store who your users are.
- Every screen and every request checks who is asking, and what they may do.
- A test account ready for the demo.

Two ways to log in

	USERNAME AND PASSWORD	OAUTH: “SIGN IN WITH GITHUB / GOOGLE”
Who checks the password	you	the provider
What you store	a hash of the password, never the password	an ID from the provider
A secret you now hold	your users’ passwords: a leak hurts them wherever they reused it	only your app’s own key
Your work	hashing, password reset, lockout	register your app, handle the redirect

OWASP Password Storage Cheat Sheet: passwords “should never be stored in plain text”; Argon2id first. OAuth 2.0: RFC 6749, 2012.

A real interface

WHAT IT'S FOR

- The audience sees only the screen. It should look finished.
- It works on a phone: the audience scores on their phone.
- Several screens and a path through them, instead of one page or a terminal.

WHAT IT ADDS

- Another language and toolchain: JavaScript, packages, a build step.
- The screen and the logic split apart, often with an API between them.
- Design decisions: layout, wording, what an empty or failed screen shows.
- For a phone app: an app store account and a review, which take time.

Three ways to build it

	IN ITS OWN WORDS	FITS
React	“a JavaScript library for rendering user interfaces”	interactive apps with many screens
Astro	“The web framework for content-driven websites”, “Zero JavaScript, by default”	sites, reports, portfolios
React Native	“brings the best parts of developing with React to native development”	phone apps, iOS and Android

react.dev, astro.build, reactnative.dev (October 2026).

SESSION 7 IDEAS

Connect it to AI: MCP

WHAT IT'S FOR

- Your program becomes something an AI assistant can use: Claude, ChatGPT, Cursor, VS Code.
- Your data and your functions, through a chat: "What did I spend on food in October?"

WHAT IT ADDS

- Each function described as a tool: a name, its inputs, what it returns.
- A server, on your machine or online.
- Whatever you expose, the AI can call. Expose only what is safe.

"MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems."
modelcontextprotocol.io, October 2026.

And more

IDEA	WHAT IT GIVES	WHAT IT ADDS
Put it online	anyone can try it on their own phone	hosting; free tiers that sleep or expire
A language model inside	summarise, classify, answer in plain words	a cost per call, an API key, user data leaving the machine
Export and share	a PDF, a CSV, a link someone can open	formats, and who can see the link
Run on a schedule	fresh data every day or every week	a scheduler such as GitHub Actions; failures nobody sees
Your own API	other programs can use yours	keys, limits, and not breaking the people who rely on it
Real or bigger data	real numbers, real surprises	cleaning, licences, and possibly personal data

For games: a player that plays itself

WHAT IT'S FOR

- A computer opponent that makes real decisions.
- A bot that tests your game by playing it, over and over.
- Each turn: the game state and the legal moves go in, one move comes out.

WHAT IT ADDS

- A decision model such as **Jev**: it writes no text, it picks one of the options you give it.
- An API key, and a cost for every move.
- A correct list of legal moves: the model only chooses from what you offer.
- A fallback move when the network fails.

Jev, TypeSafe AI, released 15 September 2026: answers with one of up to 255 options, a level, or yes/no with a probability (aimlapi.com model page; Towards AI, "What is Jev"). Check access and price before you build on it.

Two weeks on your project

Two weeks

No class until 10 November. It looks like a lot of time. It isn't: use it.

Decide what you still want to build. Pick something big enough that you might need my help.

Write the plan before you start. Your spec is the plan: a change with OpenSpec propose, and its `tasks.md` is the list. What you tick off is your progress.

Keep the demo working

First a thin path from the input to the screen, end to end. Then the details.

After every change, walk through your demo. `main` is never broken: changes go on a branch.

"Walking skeleton": Alistair Cockburn. "Tracer bullets": Andrew Hunt and David Thomas, The Pragmatic Programmer, 1999.

Track where you are

One push a day, one line in your log.

Once a week, compare with the plan. If you're behind, cut: must · should · could · won't. You cut from could.

MoSCoW prioritisation: Dai Clegg, 1994, part of the DSDM method.

Working with the agent

One task, one session. End it with a commit and a “next step” line in your log. Start the next session by having the agent read it.

Read the diff. Don’t let the agent build past your spec.

Long contexts make models worse, not only more expensive: Liu et al., “Lost in the Middle”, 2023; Chroma, “Context Rot”, 2025.

If you don’t know what to work on

Write to me.

gstack

Y Combinator

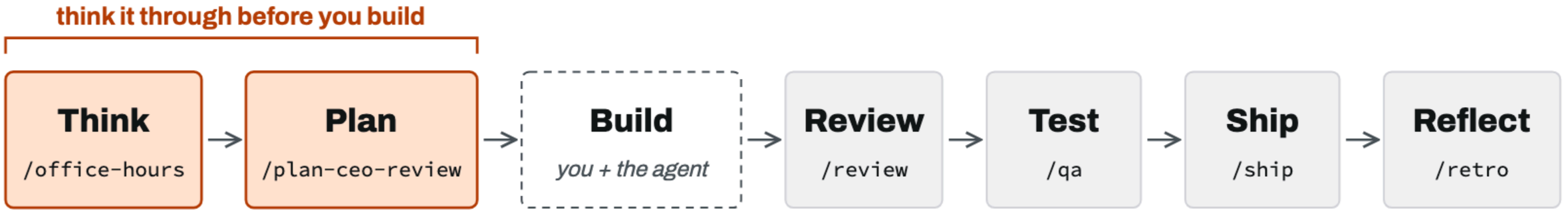
Since 2005, it has invested in very young startups: today **$500k per company**. Airbnb, Stripe and Dropbox started there.

At YC, **office hours** are when a partner sits down with the founders and asks the hard questions about their product.

ycombinator.com/about (October 2026); founders: Paul Graham, Jessica Livingston, Trevor Blackwell, Robert Morris. Airbnb, Dropbox, Stripe as YC companies: TechCrunch, 16 July 2014.

gstack

By Garry Tan, President and CEO of Y Combinator.
Open source, MIT licence, free. “It turns Claude Code into a virtual engineering team.”



github.com/garrytan/gstack, README (version 1.91, October 2026): “The sprint”.

Which command, when

IF THIS IS YOUR PROBLEM	TRY
“I’m not sure this is worth building.”	`/office-hours`
“My plan is too big for the time I have.”	`/plan-ceo-review`, Reduction mode
“It runs, but is it right?”	`/review`
“Does my demo still work after the last change?”	`/qa`
“Something broke, and I don’t know why.”	`/investigate`

Installing gstack

Open Claude Code and say:

> **Install gstack from github.com/garrytan/gstack, following its README.**

The README's own words: "paste this. Claude does the rest."

What it needs: Git and Bun (on Windows also Node.js). Setup builds its tools and downloads a browser for `/qa`. **Telemetry is off unless you say yes;** it asks once.

github.com/garrytan/gstack, README: Install, Privacy & Telemetry.

Impeccable

Impeccable

By Paul Bakaus. Open source, Apache 2.0 licence.

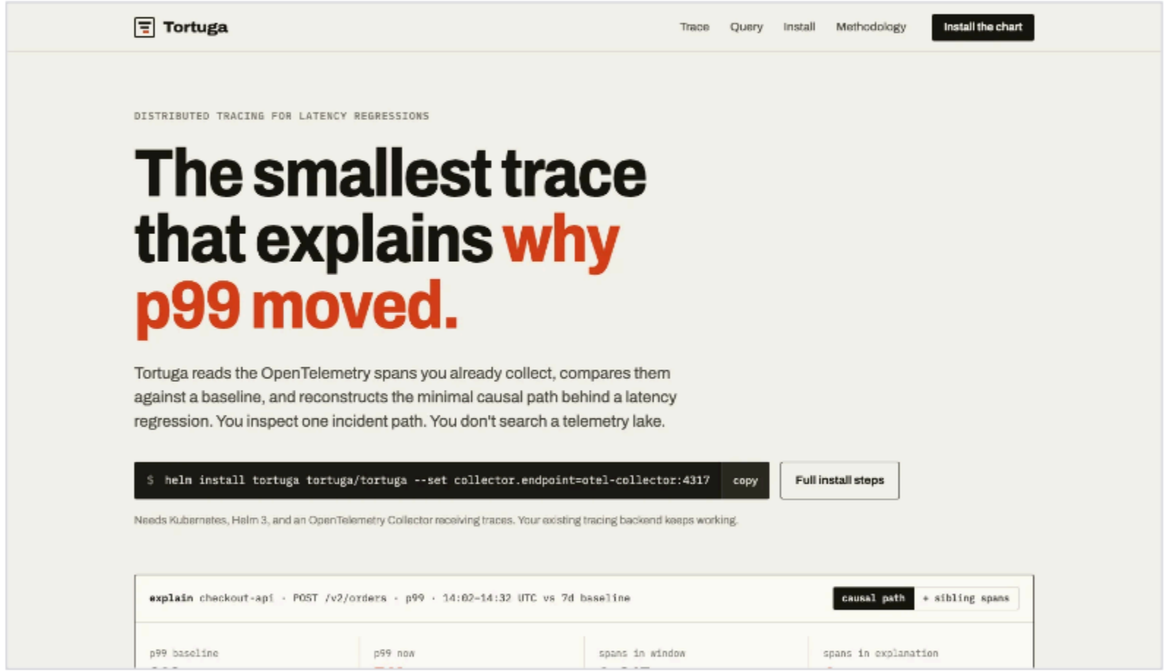
A design skill for your coding agent: Claude Code, Codex, Cursor, Gemini CLI, GitHub Copilot and others.

The audience scores whether they get it. What they see on the screen is all they have.

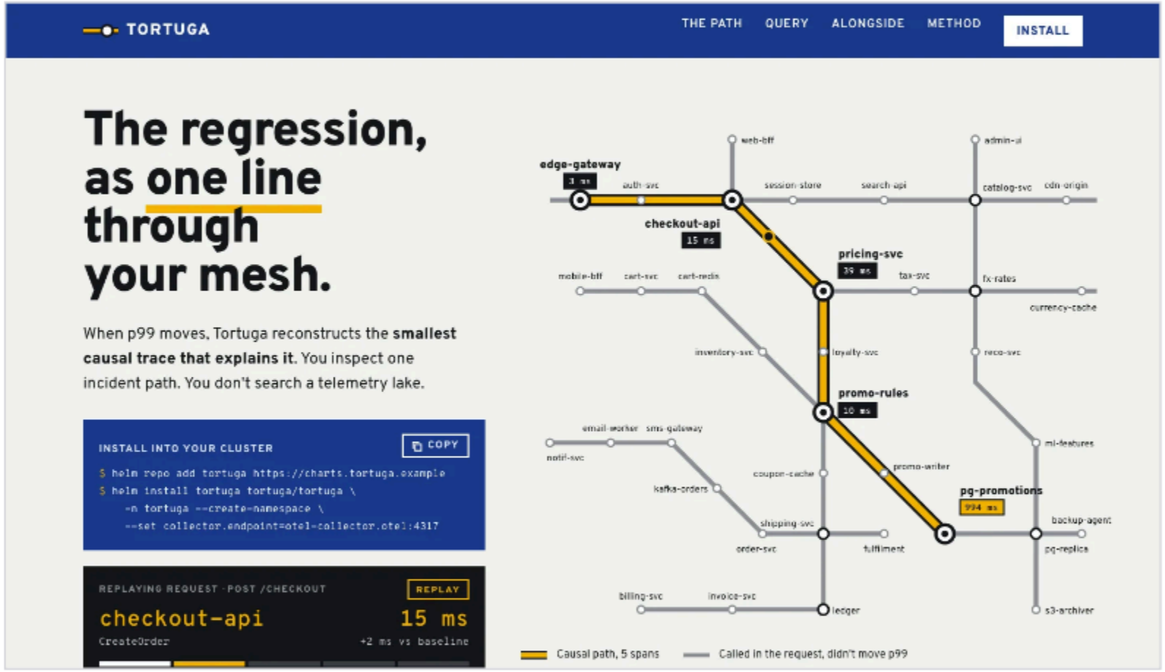
It works on your demo slides too, if you make them in HTML.

impeccable.style; the skill's own SKILL.md (version 4.6).

Same request, first pass



Without Impeccable



With Impeccable

impeccable.style/gallery, “Observability”: “First-pass results, before manual polish.”

Which command, when

IF THIS IS YOUR PROBLEM	TRY
“Something looks off, and I can’t say what.”	`critique`
“It looks wrong on a phone.”	`adapt`
“People don’t get the labels or the error messages.”	`clarify`
“It breaks on empty or strange input.”	`harden`
“Can everyone use it, and is it fast enough?”	`audit`
“I’d like to see a few versions before I choose.”	`generate`
“It works, but nobody will remember it.”	`delight`
“It works. Now it should look finished.”	`polish`

Installing Impeccable

Open Claude Code and say:

> **Install Impeccable from impeccable.style.**

It works with Codex, Cursor, Gemini CLI and GitHub Copilot as well.

impeccable.style, Install.

Homework

Homework 7 — Your two weeks

1. **What you moved forward.** What works now that did not work two weeks ago? Link the commits: what is not pushed does not exist.
2. **One question.** About your project, your demo, or anything in the course.

Due **Saturday 7 November, 20:00**. Late until Monday 9 November, 20:00, for half credit.

Homework 8 — Your demo

1. Read section 1 of your README again: The demo.
2. If it no longer describes what you want to show, change it with OpenSpec propose.
3. Commit and push.
4. Paste the final version on Canvas.

Due **Saturday 7 November, 20:00**. Late until Monday 9 November, 20:00, for half credit.

The dates

27 October, 3 November	no class
7 November	your two weeks, and your demo description
10 November	next class
28 November	your slides in your repository
1 and 8 December	demos